

INFORME INDIVIDUAL DE TRABAJO

Robert Altamar · Módulo de Lógica de Peticiones y Control de Conflictos · Proyecto g_agenda

Asignatura: Programación — Grupo: ROSIJO — Fecha: 21 de julio de 2026

1. Mi parte en el proyecto (qué he hecho)

Mi responsabilidad fue la parte de **Lógica y Conflictos**: la etapa que recibe todas las peticiones de reserva, decide cuáles son válidas, en qué orden se procesan, y resuelve qué pasa cuando dos piden el mismo hueco. Trabajé sobre las clases **Peticion** y **ProcesadorAgenda**, y me encargué de estos bloques:

- **Lectura y modelo de datos:** clase `Peticion` (actividad, espacio, fechas, máscaras) y lectura de `petitions.txt` línea a línea con Scanner.
- **Validación:** descarto peticiones con fecha de fin anterior al inicio, y peticiones con franjas horarias mal formadas (p. ej. "23-02").
- **Orden y prioridad:** las peticiones "Tancat" (cierre de sala) se procesan siempre primero, y el resto se agrupa por actividad.
- **Desglose de máscaras:** programé el motor que rompe cadenas como "08-10_14-16" usando `split` en dos niveles ("_" separa bloques, "-" separa hora de inicio y de fin) para traducirlas a días y horas concretas.
- **Matriz de ocupación:** una matriz de Strings de 24×32 por cada sala (24 horas, días del mes), donde cada celda representa una hora concreta de un día concreto.
- **Control de colisiones:** compruebo si una celda ya está ocupada antes de aceptar una petición nueva; gana siempre la primera que llega, según el orden de prioridad ya establecido.
- **Persistencia de incidencias:** vuelco los conflictos detectados al fichero `incidencias.log` con `FileWriter` y `PrintWriter`, junto con el recuento de horas asignadas y no asignadas por actividad.

2. Lo que pulimos y lo más complicado

El troceado de las máscaras dobles. Lo más laborioso fue el parseo en dos niveles: una misma petición puede traer varios tramos horarios en la misma línea ("13-14_17-18"). Tuve que asegurarme de que cada tramo se convertía en las horas correctas sin dejarme ninguna ni colarme en la siguiente (por ejemplo, "08-10" debe dar las horas 8 y 9, nunca la 10).

El posible desacuerdo léxico con el resto del equipo. En la revisión conjunta detectamos que mi código compara literalmente con la palabra "Tancat", tal como aparece en nuestro `petitions.txt` de pruebas, pero existe el riesgo de que el fichero de idiomas de Jose Antonio o la versión final de datos use "Cerrado". Como esta comparación no da ningún error si falla (simplemente deja de funcionar la prioridad de cierre), quedó pendiente de confirmar con el grupo qué palabra exacta se usará en todos los ficheros y en todo el código.

La prioridad de los bloqueos. Tuve que decidir un orden claro: una petición de cierre ("Tancat") gana siempre frente a cualquier otra reserva, y ninguna otra petición puede sobrescribir una celda ya ocupada. Traducir esa regla a un if/else estricto, y comprobarlo con datos reales (incluida una prueba donde dos peticiones de cierre distintas chocaban entre sí en domingo), me llevó varias vueltas.

3. Lo que me gustó y lo que me costó

Lo que me gustó. El control matricial. Ver cómo el sistema denegaba automáticamente una reserva que se solapaba con otra ya existente, y dejaba constancia de ello en un archivo de log con el nombre exacto de quién ganó y quién perdió el hueco, fue muy satisfactorio: es lógica de negocio real, del tipo que hay detrás de cualquier sistema de reservas.

Lo que me costó. Diseñar el control de colisiones de forma estricta (usar == null para detectar celdas libres, en vez de comparar textos) y gestionar la escritura del fichero de incidencias sin perder ninguna coordenada. Trabajar a la vez con la matriz, las listas de días/horas y los contadores de horas asignadas/no asignadas fue lo que más concentración me exigió.

4. Mi aporte al equipo

Fui la etapa intermedia del flujo: recibo la configuración y las herramientas de fecha/idioma que prepara Jose Antonio, aplico las reglas de validación, orden y conflictos del enunciado, y entrego a Simón una matriz ya resuelta, lista para dibujar en HTML. Sin esta capa lógica, el frontend no tendría datos coherentes que representar, y no habría ningún registro de qué reservas fallaron ni por qué.

5. Conclusión

Desarrollar el motor lógico me obligó a profundizar en el manejo de matrices, en el troceado de cadenas en varios niveles y en la escritura de ficheros con FileWriter/PrintWriter, temas que al principio se me hacían abstractos. El mayor aprendizaje ha sido diseñar un sistema de validación y control de colisiones con prioridades claras, verificando cada pieza por separado antes de unirla al resto. Me llevo, sobre todo, la importancia de consensuar los detalles con el equipo: un simple desacuerdo entre "Tancat" y "Cerrado" puede tumbar en silencio toda la lógica de prioridad, sin que ningún error avise de ello.